

EMBEDDED SYSTEM MICROSERVICES PROJECT

Detailed Design Document

Team: DCC-6111

Client: SLB

Ikir Bejleri, Ahmed Alamin, Shane Hanley, Berk Tunctan, Daniel Thomson

Repository: <https://github.com/ahmedalaminn/dcc-6111>

Georgia Institute of Technology
CX 3803 — Junior Design Capstone
Spring 2026

Table of Contents

<u>Table of Figures</u>	3
<u>Terminology</u>	4
<u>Introduction</u>	6
<u>Background</u>	6
<u>Document Summary</u>	6
<u>Static System Architecture</u>	7
<u>Goals and Objectives</u>	7
<u>Rationale</u>	7
<u>Static Architecture Diagram</u>	7
<u>Dynamic System Architecture</u>	10
<u>Sequence Diagram</u>	10
<u>Component Detail and Consistency</u>	11
<u>Explanation</u>	12
<u>Data Storage Design</u>	13
<u>Waveform Monitor — File-Based Storage</u>	13
<u>PubSub Network Logger — In-Memory State</u>	16
<u>Git Fork Monitor — API-Driven, No Local Persistence</u>	16
<u>Data Exchange</u>	16
<u>UI Design and Heuristic Analysis</u>	18
<u>Waveform Monitor</u>	18
<u>PubSub Network Logger</u>	19
<u>Git Fork Monitor</u>	20
<u>Heuristic Analysis</u>	20
<u>Design Considerations</u>	21
<u>Appendix</u>	22
<u>Team Member Contributions</u>	22

Table of Figures

Figure 1 — Static System Architecture: Component Dependency Diagram	8
Figure 2 — Data Storage Directory Structure	13
Figure 3 — WAVE v1 Binary Header Layout	14
Figure 4 — ZeroMQ Publish-Subscribe Network Topology	16
Figure 5 — Dynamic System Architecture: Waveform Sequence Diagram	11
Figure 6 — Waveform Monitor Dashboard	18
Figure 7 — PubSub Network Logger Dashboard	19
Figure 8 — Git Fork Monitor Scan Results	20

Terminology

Term	Definition	Context
API	Application Programming Interface: a set of functions and protocols for building software.	REST endpoints exposed by Flask services
FFT	Fast Fourier Transform: an algorithm to compute the discrete Fourier transform efficiently.	Used to analyze waveform frequency content
SNR	Signal-to-Noise Ratio: the ratio of signal power to noise power, measured in decibels.	Key metric in waveform quality assessment
RMS	Root Mean Square: the square root of the mean of squared sample values.	Measures effective amplitude of a waveform
RMSE	Root Mean Square Error: the square root of the mean squared difference between two signals.	Quantifies divergence between waveforms
ZeroMQ	A high-performance asynchronous messaging library for distributed systems.	PubSub backbone for inter-node communication
X PUB/XSUB	Extended publish/subscribe socket types in ZeroMQ that allow subscription forwarding through a broker.	Used in the PubSub Network Logger broker
SSE	Server-Sent Events: a standard for pushing real-time updates from a server to a browser over HTTP.	Used by the PubSub Logger web dashboard
Protobuf	Protocol Buffers: a language-neutral binary serialization format.	Defines the LogMessage schema for PubSub traffic
BBB	BeagleBone Black: an ARM Cortex-A8 single-board computer with 512 MB RAM.	Target embedded deployment platform
Next.js	A React-based full-stack web framework with server-side rendering.	Powers the Git Fork Monitor web application
NextAuth	An authentication library for Next.js that supports OAuth providers.	Handles GitHub OAuth login in the Fork Monitor

Octokit	The official GitHub REST API client library.	Used to query repositories, forks, and commits
Vite	A fast frontend build tool for modern web applications.	Bundles the PubSub Logger React dashboard
Flask	A lightweight Python web microframework.	Serves the Waveform Monitor and PubSub Logger APIs
CLI	Command-Line Interface.	Waveform Monitor supports a full CLI for scripted workflows
ADC	Analog-to-Digital Converter: hardware that converts analog signals to digital samples.	Source of raw uint8 waveform captures on the BBB

Introduction

Background

The SLB Embedded System Microservices Project is a capstone effort for CX 3803 at Georgia Tech. The client, SLB (formerly Schlumberger), requires a set of internal engineering tools to support their embedded development workflow. The project delivers three independent tools: a Waveform Monitor and Comparator for ingesting and analyzing binary waveform data, a ZeroMQ-based PubSub Network Logger for real-time diagnostic telemetry across up to five networked nodes, and a Git Fork Monitor for tracking framework adoption and code drift across GitHub repository forks. The Waveform Monitor and PubSub Logger are designed to run on resource-constrained embedded hardware such as the BeagleBone Black (ARM Cortex-A8, 512 MB RAM). The Git Fork Monitor is a separate web application intended for use by software engineers on standard developer workstations.

Because the tools are designed for internal engineering use, they prioritize minimal dependencies, low memory usage, and file-based persistence over features like user authentication or external-facing security hardening. The three tools are fully independent: they share no runtime state, no database, and are not deployed to the same device.

Document Summary

This Detailed Design Document describes the internal architecture and data design of the system. It is organized into the following sections:

Static System Architecture: Presents the high-level component structure and dependency relationships for all three tools.

Dynamic System Architecture: Provides a sequence diagram illustrating the runtime behavior of the Waveform Monitor during ingestion, comparison, and report generation.

Data Storage Design: Documents how each tool stores, formats, and exchanges data.

UI Design and Heuristic Analysis: Presents the primary screens of each tool with descriptions and a heuristic evaluation.

Static System Architecture

Goals and Objectives

The project delivers three separate tools for SLB engineers. Each tool is independently deployable and operates without shared state. The primary objectives are: (1) ingest, analyze, and compare binary waveform captures with computed signal metrics such as SNR, FFT, and cross-correlation (Waveform Monitor, targeting BBB); (2) provide real-time diagnostic telemetry logging across a ZeroMQ publish-subscribe network of up to five nodes (PubSub Logger, targeting BBB); and (3) enable framework drift detection and fork alignment analysis across GitHub repository hierarchies (Git Fork Monitor, running on developer workstations).

Rationale

The architecture follows a layered design pattern within each tool. This separation keeps binary parsing, signal processing, data persistence, and presentation concerns independent of one another, which simplifies testing and allows individual layers to be replaced without affecting the rest of the stack. Flask was chosen as the web framework for both the Waveform Monitor and the PubSub Logger because it is lightweight, has minimal dependencies, and runs well on ARM-based embedded Linux. Next.js was selected for the Git Fork Monitor because it provides server-side rendering and built-in API routes, which simplifies the OAuth flow with GitHub. The Git Fork Monitor runs on standard developer machines and does not need to meet the same resource constraints as the embedded tools.

ZeroMQ was chosen over HTTP-based messaging for the PubSub Logger because it provides low-latency, broker-mediated publish-subscribe semantics that work well for real-time telemetry on a local network. The XPUB/XSUB proxy pattern decouples publishers from subscribers and allows the broker to forward subscription filters, which keeps network traffic efficient.

No external database is used by any of the three tools. The Waveform Monitor uses a directory-based file storage model with CSV metric logs and JSON comparison results. The PubSub Logger holds all state in memory because its data is transient diagnostic telemetry. The Git Fork Monitor queries the GitHub API on demand and stores no persistent data beyond the user's session token. For the two embedded tools, this file-and-memory approach eliminates the need to install and maintain a database server on the target hardware.

Security considerations are minimal because all three tools are internal engineering utilities. The Waveform Monitor and PubSub Logger are deployed on a private local network and do not require user authentication. The Git Fork Monitor uses GitHub OAuth exclusively to obtain a scoped API token; no credentials are stored locally beyond the NextAuth session JWT.

Static Architecture Diagram

The following component diagram shows the major components of all three tools and their dependency relationships. Solid arrows indicate dependency. The user is not shown because this is a static view.

Figure 1: Static System Architecture — Component Dependency Diagram

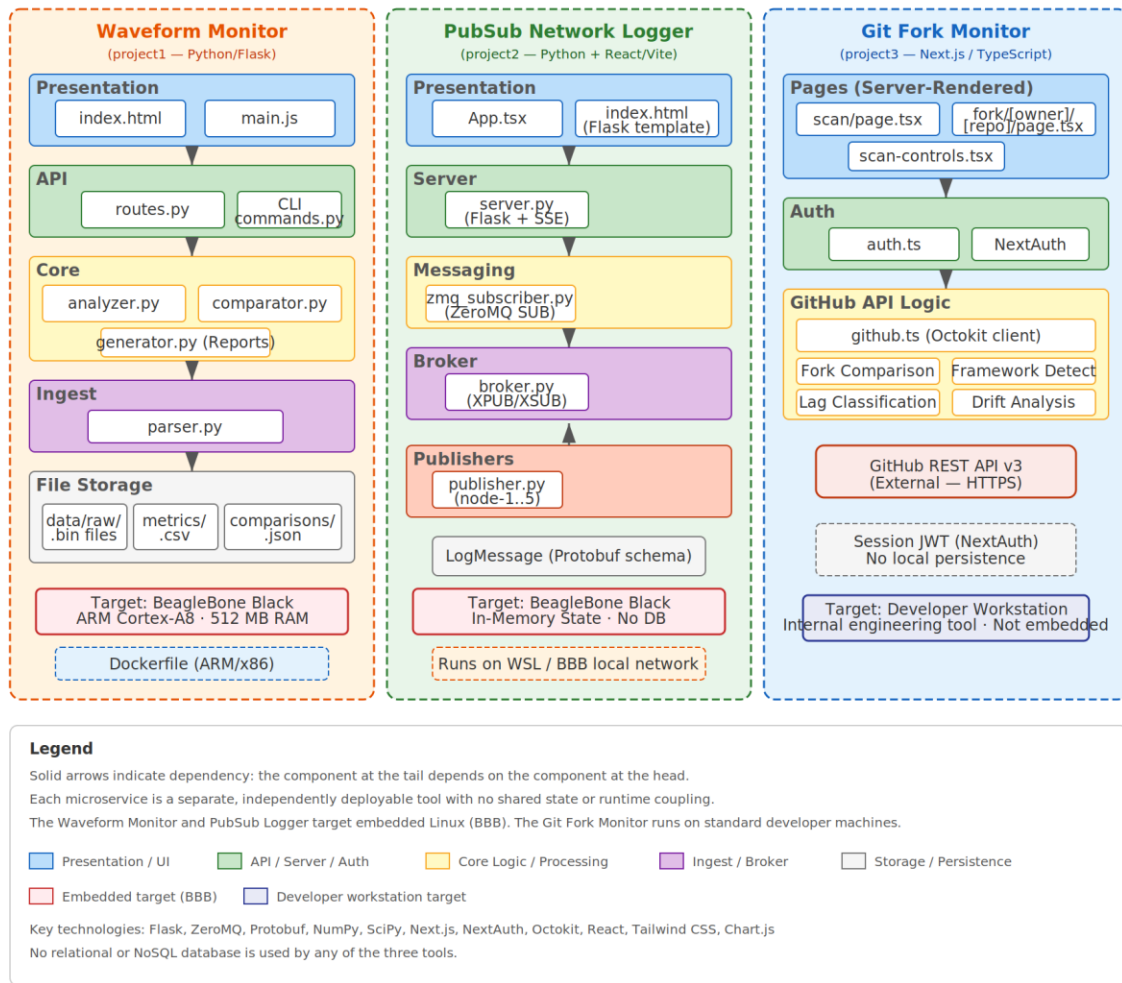


Figure 1: Static System Architecture — Component Dependency Diagram

The Waveform Monitor follows a four-layer architecture: the Ingest Layer (parser.py) decodes binary files into a normalized WaveformData representation; the Core Layer (analyzer.py, comparator.py) performs all signal processing; the API Layer (routes.py) exposes REST endpoints; and the Presentation Layer (index.html, main.js) renders the web UI. The CLI module (commands.py) provides an alternative entry point that bypasses the API layer and calls the Core and Ingest layers directly. The Report Generator (generator.py) reads from the file-based storage layer and produces HTML engineering reports using Jinja2 templates.

The PubSub Network Logger uses a broker-based topology: a ZeroMQ XSUB/XPUB proxy (broker.py) sits between publisher nodes (publisher.py) and the Flask server (server.py). The server's ZmqSubscriber thread receives deserialized LogMessage objects and pushes them to connected browser clients via Server-Sent Events. The React frontend (App.tsx) consumes the SSE stream and renders a real-time log table with per-node status indicators.

The Git Fork Monitor is a standalone Next.js application that runs on developer workstations, not on embedded hardware. The auth module (auth.ts) configures GitHub OAuth via NextAuth. The

github module (github.ts) contains all GitHub API logic, including repository listing, fork comparison, commit distance estimation, framework detection, and lag classification. Server-rendered pages (scan/page.tsx, fork/[owner]/[repo]/page.tsx) call the github module at request time and return fully rendered HTML with Tailwind CSS styling.

Dynamic System Architecture

This section zooms into the component level and illustrates a non-trivial runtime interaction. The scenario presented is the core workflow of the Waveform Monitor: an engineer uploads a binary waveform file, the system parses, analyzes, and stores it, then the engineer compares two waveforms and generates an engineering report. This three-phase sequence exercises every layer of the Waveform Monitor's architecture and demonstrates how the static components identified in the previous section interact at runtime.

Sequence Diagram

The sequence diagram below shows the interactions between six components: the Engineer (actor), the Web UI (index.html + main.js), the API layer (routes.py), the parser (parser.py), the signal processing engine (analyzer.py / comparator.py), and the file-based storage layer. The scenario is divided into three phases.

Phase 1 — Waveform Ingestion: The engineer uploads a .bin file through the Ingest tab. The Web UI sends a multipart POST to /api/ingest. The API layer calls parser.py to decode the binary file into a WaveformData object, then passes the samples to analyzer.py to compute metrics (SNR, FFT, RMS, peak-to-peak, dominant frequency). The raw file is saved to data/raw/<source_id>/ and the scalar metrics are appended to the source's CSV in data/metrics/. The API returns the computed metrics as JSON, and the Web UI renders them as tiles.

Phase 2 — Waveform Comparison: The engineer selects two waveforms in the Compare tab and submits. The API loads both files through parser.py, then calls compare_signals in analyzer.py, which performs cross-correlation to align the signals, computes RMSE and Pearson correlation, and runs detect_degradation to flag any significant changes (amplitude shift, SNR drop, frequency drift). The comparison result is saved as JSON in data/comparisons/, and the Web UI renders waveform overlays, delta plots, and FFT spectrum comparisons.

Phase 3 — Report Generation: The engineer clicks "Generate Report." The API calls generator.py, which reads the source's metrics CSV and any referenced comparison JSON, renders them into an HTML report using the Jinja2 template, and saves the output to data/reports/. The Web UI provides a link to view the report.

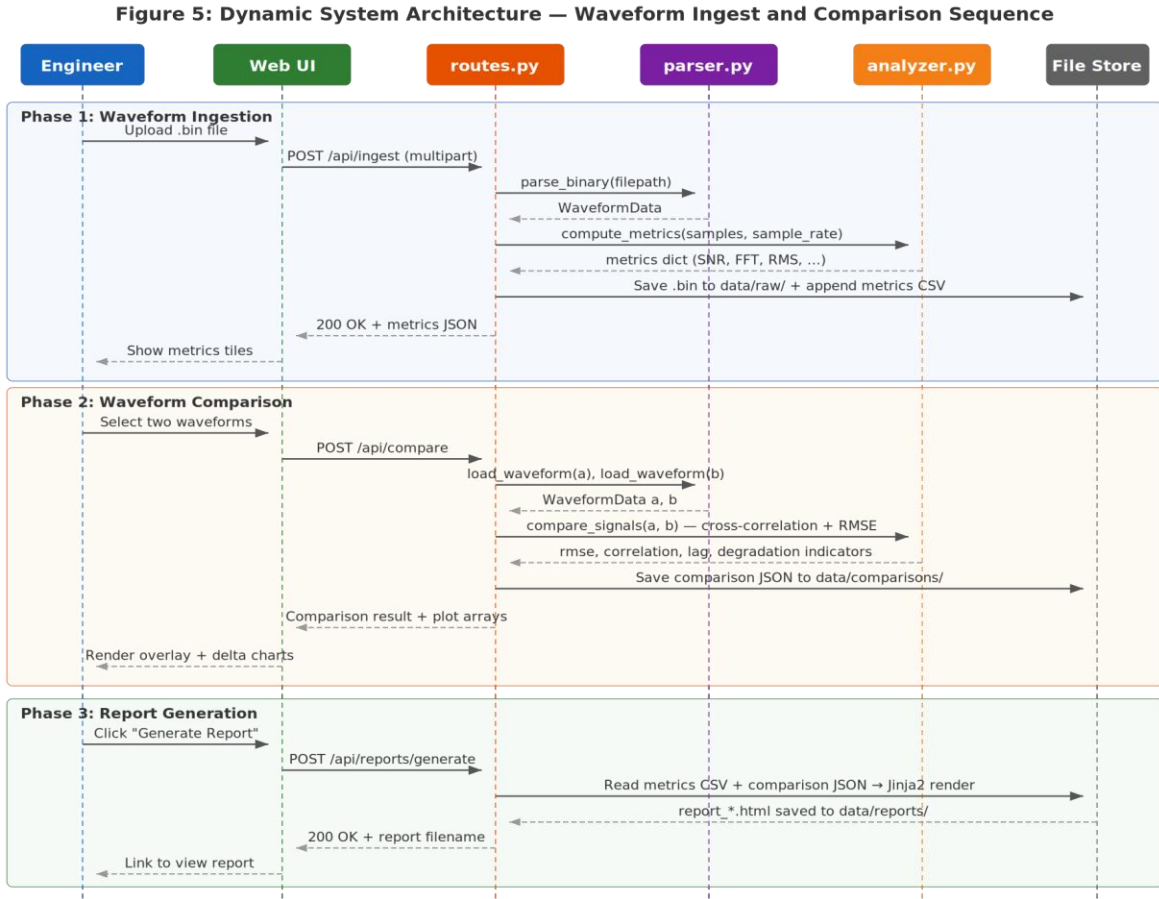


Figure 5: Dynamic System Architecture — Waveform Ingest and Comparison Sequence

This sequence demonstrates key architectural properties: the parser is isolated from the analyzer, so the binary format can change without affecting signal processing; the comparator reuses the same `compute_metrics` function used during ingestion, ensuring metric consistency; the file storage layer is a passive sink, and the API layer coordinates all reads and writes; the Web UI never accesses storage directly.

Component Detail and Consistency

The sequence diagram in Figure 5 zooms into the same components shown in the static architecture diagram (Figure 1) and preserves their layered relationships. The six participants in the sequence (Web UI, `routes.py`, `parser.py`, `analyzer.py/comparator.py`, and File Storage) map directly to the Presentation, API, Ingest, Core, and File Storage layers of the Waveform Monitor column in Figure 1. No component in the sequence diagram communicates with a layer it does not depend on in the static diagram: the Web UI never calls `parser.py` directly, and the file storage layer is only accessed by the API layer and the report generator. This correspondence between the static and dynamic views confirms that the layered architecture is maintained at runtime and not just structural convenience.

The `compare_signals` function reuses the same `compute_metrics` pipeline invoked during ingestion, which means the SNR, FFT, and RMS values reported in a comparison result are

computed with the same code path as the values in the metrics CSV. This eliminates the risk of metric drift between the single-waveform and comparison workflows. Similarly, the report generator reads directly from the persisted CSV and JSON files rather than recomputing metrics, so the report always reflects exactly what was stored — not a potentially different recalculation.

Explanation

The scenario was chosen because it exercises the full depth of the Waveform Monitor's architecture in a single, realistic workflow that an SLB engineer would perform during a field validation session. The three phases — ingest, compare, report — are not independent features but a sequential pipeline where the output of each phase feeds the next: ingestion produces the stored waveform and metrics that comparison reads, and comparison produces the JSON that the report generator renders. This chained dependency makes the scenario non-trivial and demonstrates that the architecture can support multi-step workflows without bypassing layers.

Phase 1 (Ingestion) shows how a raw binary file moves through the Ingest layer for decoding and the Core layer for analysis before reaching storage. The separation matters because binary format changes (e.g., switching from WAVE v1 to a different capture format) only affect `parser.py`, while signal processing changes (e.g., adding a new metric) only affect `analyzer.py`.

Phase 2 (Comparison) demonstrates the most computationally intensive operation in the system. Cross-correlation alignment is necessary because waveforms captured at different times may have a sample offset, and computing RMSE on misaligned signals would produce meaningless results. The degradation detection step adds domain value by flagging amplitude shifts, SNR drops, or frequency drift that might indicate equipment degradation — this is the primary use case the client cares about.

Phase 3 (Report Generation) closes the loop by producing a self-contained HTML artifact that the engineer can archive or share. The Jinja2 template approach was chosen over a frontend-only export because the report needs to be viewable without the Waveform Monitor running, which is important for handoff to teams that may not have the tool installed.

Data Storage Design

This section documents how data is stored, formatted, and exchanged across all three tools. Because the Waveform Monitor and PubSub Logger are designed for resource-constrained embedded hardware, their persistence is file-based or in-memory. The Git Fork Monitor, which runs on developer workstations, queries GitHub on demand and persists nothing locally. No relational or NoSQL database is used by any of the three tools.

Waveform Monitor — File-Based Storage

The Waveform Monitor uses a directory-tree structure rooted at a configurable DATA_DIR (default: project1/data/). All waveform files, computed metrics, comparison results, and generated reports are organized into subdirectories by function.

Figure 2: Waveform Monitor — Data Storage Directory Structure

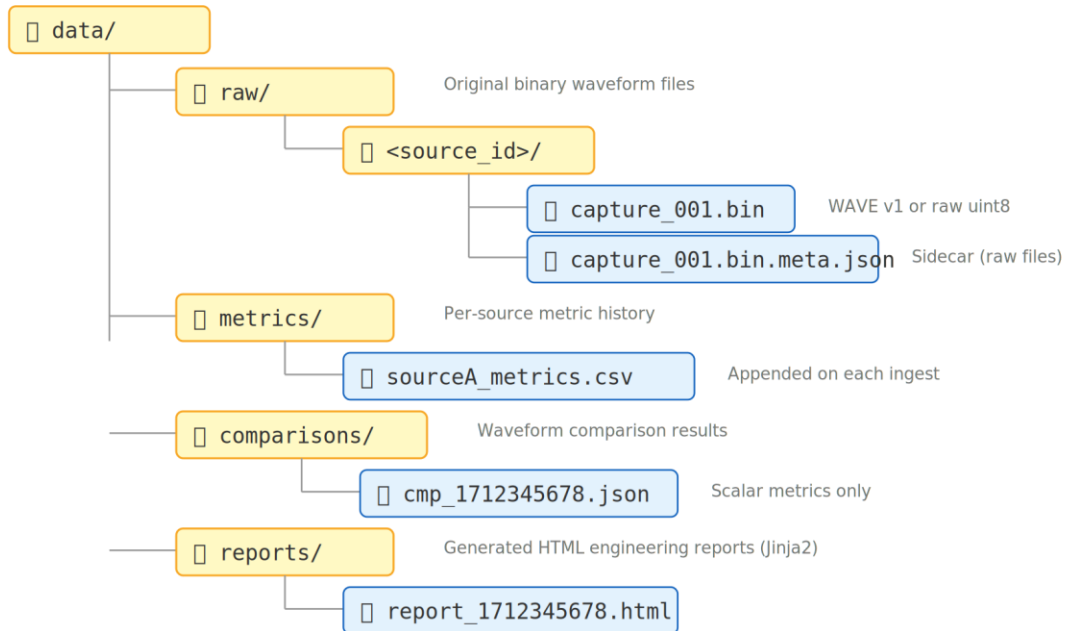


Figure 2: Data Storage Directory Structure

Directory Layout

Directory	Contents	Format
data/raw/<source_id>/	Original binary waveform captures, organized by source identifier.	.bin files (WAVE v1 or raw uint8)
data/metrics/	Per-source metric history. One CSV file per source_id, appended on each ingest.	.csv (keys: filename, timestamp_ms, peak_to_peak, max, min, rms, mean, std, snr_db, dominant_freq_hz, dominant_freq_magnitude,

		num_samples, computed_at_ms)
data/comparisons/	Waveform comparison results. Scalar metrics only (plot arrays excluded).	.json (keys: id, timestamp_ms, label_a/b, rmse, correlation, alignment_lag_samples, degradation_indicators)
data/reports/	Generated HTML engineering reports.	.html (rendered via Jinja2 from report.html template)

Raw binary waveform files that arrive without a WAVE v1 header (raw ADC captures from the BeagleBone Black) receive a companion .meta.json sidecar file that stores the sample rate, source identifier, and units.

WAVE v1 Binary Format

The Waveform Monitor uses a custom binary format called WAVE v1 for waveform files. The format is big-endian and consists of a 69-byte fixed header followed by a variable-length array of float32 samples.

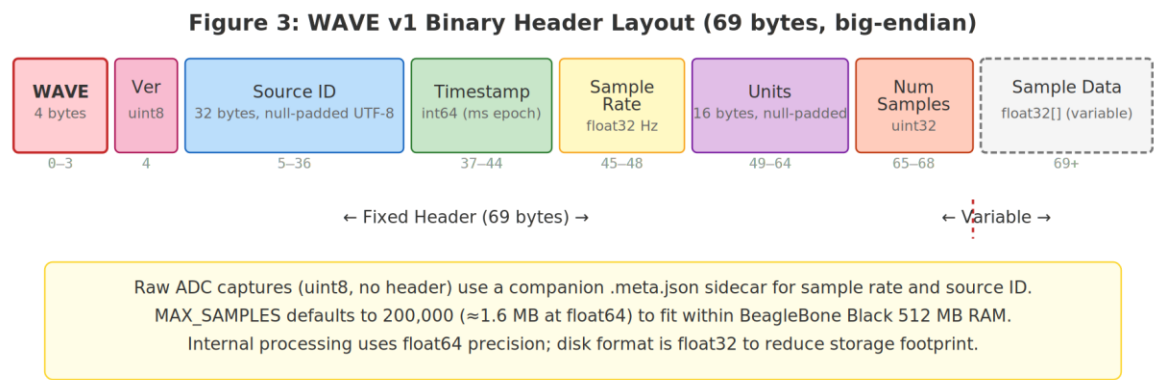


Figure 3: Data Exchange Format — WAVE v1 Binary Header Layout

Byte Offset	Field	Type	Description
0-3	Magic	4 bytes	ASCII "WAVE" — identifies the file format
4	Version	uint8	Format version (currently 1)
5-36	Source ID	32 bytes	Null-padded UTF-8 source identifier string
37-44	Timestamp	int64	Capture timestamp in milliseconds since epoch
45-48	Sample Rate	float32	Sampling rate in Hz (e.g. 200000.0)
49-64	Units	16 bytes	Null-padded UTF-8 unit label (e.g. "mV")
65-68	Num Samples	uint32	Number of float32 samples that follow
69+	Samples	float32[]	Waveform sample data in big-endian IEEE 754

The parser caps reads at MAX_SAMPLES (default 200,000) to stay within the BBB's 512 MB RAM. At float64 precision, 200,000 samples consume approximately 1.6 MB.

PubSub Network Logger — In-Memory State

The PubSub Network Logger does not persist for any data to disk. All states are held in memory for the lifetime of the server process.

State Model

State Variable	Type	Purpose
_node_last_seen	dict[str, float]	Maps each node_id to its last-seen epoch timestamp. The watchdog detects lost nodes after 10 seconds.
_logging_enabled	bool	Global toggle for diagnostic logging. When disabled, messages are still received but not broadcast.
_clients	list[Queue]	One bounded queue (maxsize=200) per connected SSE client. Full queues drop oldest events.

LogMessage Schema

Field	Type	Description
node_id	string	Unique identifier for the publishing node
timestamp_ms	int64	Message creation time in milliseconds since epoch
payload	string	Diagnostic telemetry payload
topic	string	ZeroMQ topic filter string

ZeroMQ Network Topology

The PubSub Logger uses a broker-mediated XPUB/XSUB topology. Publisher nodes connect to port 5556, and the Flask server's ZmqSubscriber connects to port 5555. This proxy pattern allows publishers and subscribers to start and stop independently.

Figure 4: ZeroMQ Publish-Subscribe Network Topology

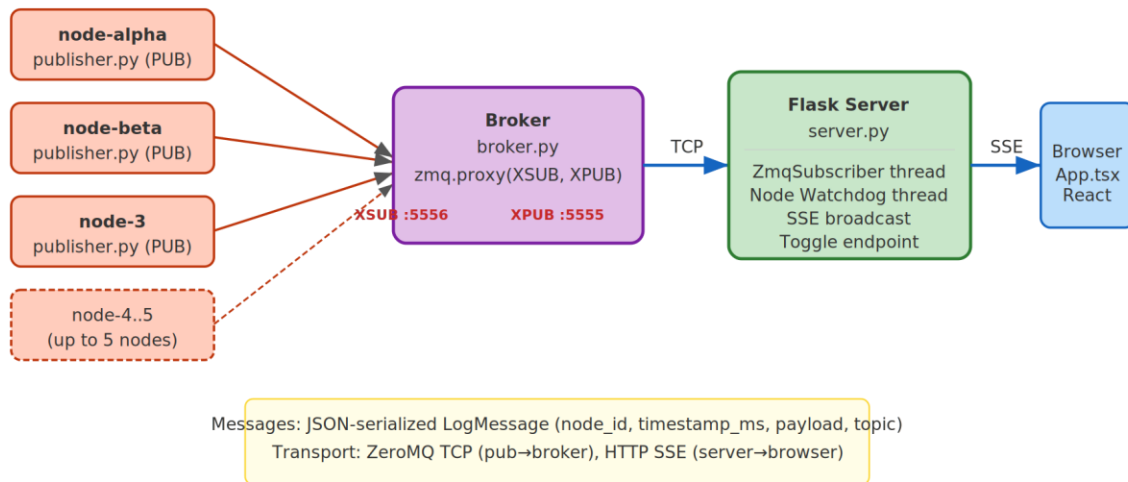


Figure 4: ZeroMQ Publish-Subscribe Network Topology

Git Fork Monitor — API-Driven, No Local Persistence

The Git Fork Monitor does not store any data locally beyond what NextAuth places in the user’s session JWT. All data is fetched from the GitHub REST API on demand using the Octokit client.

Key Data Types

Type	Source	Description
RepoWithForks	octokit.repos.listForAuthenticatedUser	Upstream repository with its forks, size, and default branch.
ForkComparison	octokit.repos.compareCommitsWithBasehead	Full diff: ahead/behind counts, file-level changes, commit lists.
ForkAlignmentSummary	Computed from ForkComparison	Fork status (Up-to-date / Slightly behind / Lagging) with configurable thresholds.
FrameworkAdoption	detectFrameworkTags (tree scan)	Framework usage across forks with weighted adoption scores and drift classification.
ProjectForkAnalysis	fetchProjectForkAnalysis	Top-level result: fork summaries, framework adoption, scan config.

Data Exchange

Formats and Protocols

Interface	Format	Protocol	Direction
Waveform Monitor REST API	JSON	HTTP (Flask, port 8000)	Browser ↔ Server
Waveform file ingest	WAVE v1 / raw uint8	HTTP multipart or CLI	File → Server
PubSub Logger telemetry	JSON-serialized LogMessage	ZeroMQ TCP (5555/5556)	Publisher → Broker → Sub
PubSub Logger dashboard	JSON	HTTP + SSE (port 5000)	Server → Browser
Git Fork Monitor API	JSON	HTTPS (GitHub API v3)	Server → GitHub
Git Fork Monitor auth	OAuth 2.0 token	HTTPS (NextAuth)	Browser ↔ GitHub ↔ Server

Security Considerations

All three tools are internal engineering utilities. The Waveform Monitor and PubSub Logger are deployed on a private local network. The Git Fork Monitor is accessed by engineers on their workstations.

Encryption: No data-at-rest encryption. ZeroMQ traffic is unencrypted TCP on the local network. The Git Fork Monitor communicates with GitHub over HTTPS.

Authentication: The Waveform Monitor and PubSub Logger do not require login. The Git Fork Monitor authenticates via GitHub OAuth (read:user, user:email, repo scopes). The access token is stored only in the server-side JWT.

PII: No PII is collected or stored. The Git Fork Monitor displays GitHub usernames as returned by the API but does not persist for them.

Input validation: File uploads are constrained by MAX_FILE_SIZE_MB (100 MB) and validated against WAVE v1 magic bytes. The Git Fork Monitor validates all scan config inputs with range checks.

UI Design and Heuristic Analysis

This section presents the primary user interface screens of each tool and evaluates them against Nielsen's ten usability heuristics. Since all three tools are internal engineering utilities, the interfaces prioritize information density and functional clarity over visual polish.

Waveform Monitor

The Waveform Monitor presents a dark-themed, single-page dashboard accessed at <http://localhost:8000>. The layout consists of a sidebar for source and waveform selection, and a main content area organized into seven tabs: Waveform, FFT Spectrum, Metrics Trend, Compare, Ingest, Reports, and Terminal.

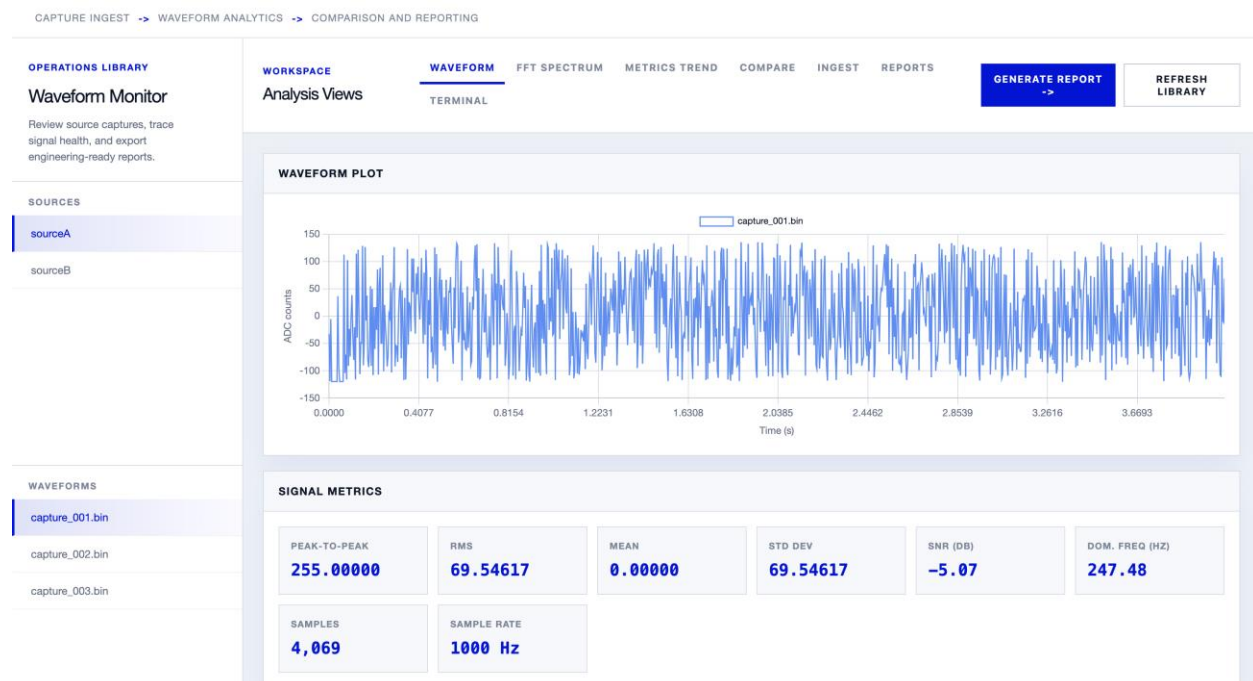


Figure 6: Waveform Monitor Dashboard — Waveform tab with signal metrics

The main screen (Figure 6) shows the waveform plot rendered with Chart.js and a row of signal metric tiles below it. The sidebar lists all ingested sources with a green indicator for the selected source and its associated .bin files. The “Generate Report” button in the top-right triggers a one-click HTML report covering the selected source’s metrics and comparisons.

Heuristic Analysis

Visibility of System Status: The status bar at the bottom always shows the number of loaded sources and the currently selected waveform file. Metric tiles update immediately after selection, giving continuous feedback.

Recognition Rather Than Recall: All available sources and waveforms are listed in the sidebar, eliminating the need to remember file names. Tab labels describe each function clearly (Waveform, FFT Spectrum, Compare).

Aesthetic and Minimalist Design:

PubSub Network Logger

The PubSub Logger provides two parallel frontends: a Flask-served HTML template (index.html at port 5000) for headless BBB deployment, and a React/Vite dashboard (App.tsx at port 5173) for development. Both consume the same SSE /stream endpoint. The interface shows a node status row and a scrolling log table.

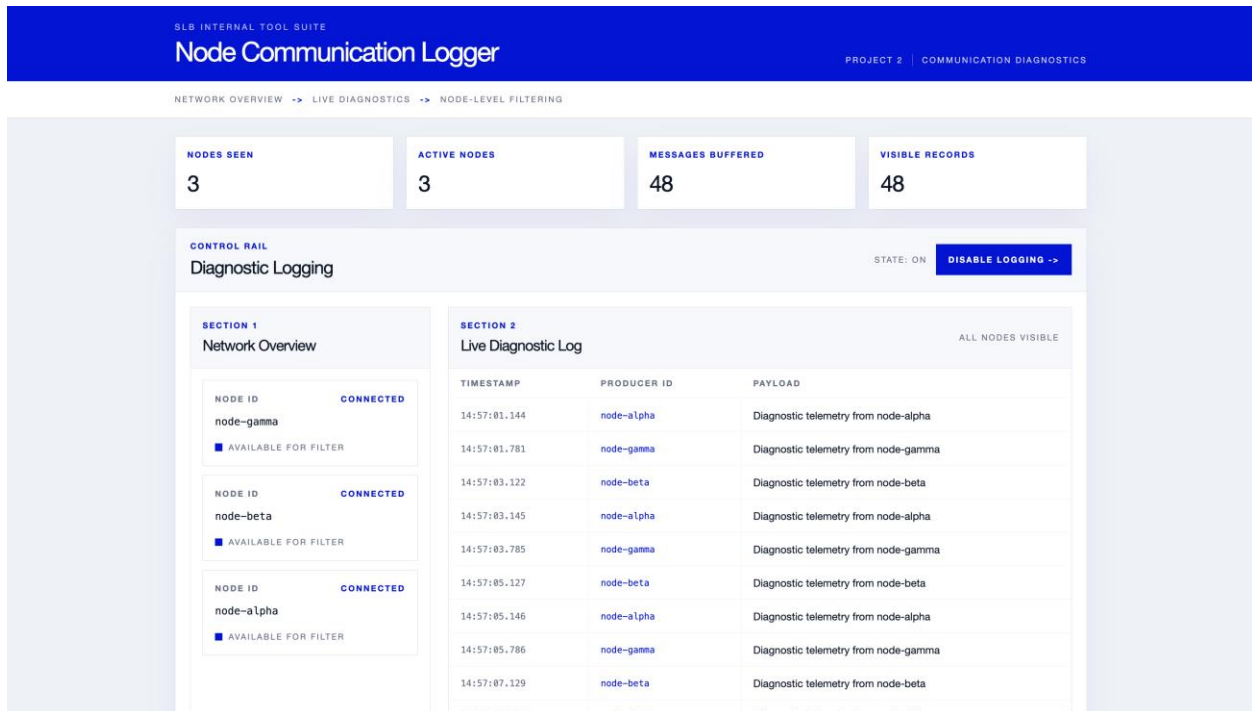


Figure 7: PubSub Network Logger Dashboard — Real-time log with node status

Figure 7 shows the logger receiving telemetry from five nodes. Each node has a colored status indicator: green for active, red for lost (no messages within the 10-second timeout). The log table displays timestamped messages with node ID, topic, and payload. The logging toggle in the top-right enables or disables log broadcasting without disconnecting from the ZeroMQ network.

Heuristic Analysis

Visibility of System Status: Node status indicators provide immediate awareness of which nodes are active or lost. The status bar shows the active node count and the total message count.

User Control and Freedom: The logging toggle allows engineers to pause log output without losing the ZeroMQ connection, so they can freeze the display to inspect a specific message without missing incoming data.

Error Prevention: The system caps the client's queue at 200 events and drops the oldest on overflow, preventing browser tab crashes from unbounded memory growth during sustained high-throughput telemetry.

Git Fork Monitor

The Git Fork Monitor is accessed through a browser after signing in with GitHub OAuth. The main scan results page displays summary cards (total repos, analyzed forks, lagging forks, scan mode) and a sortable table of fork alignment summaries.

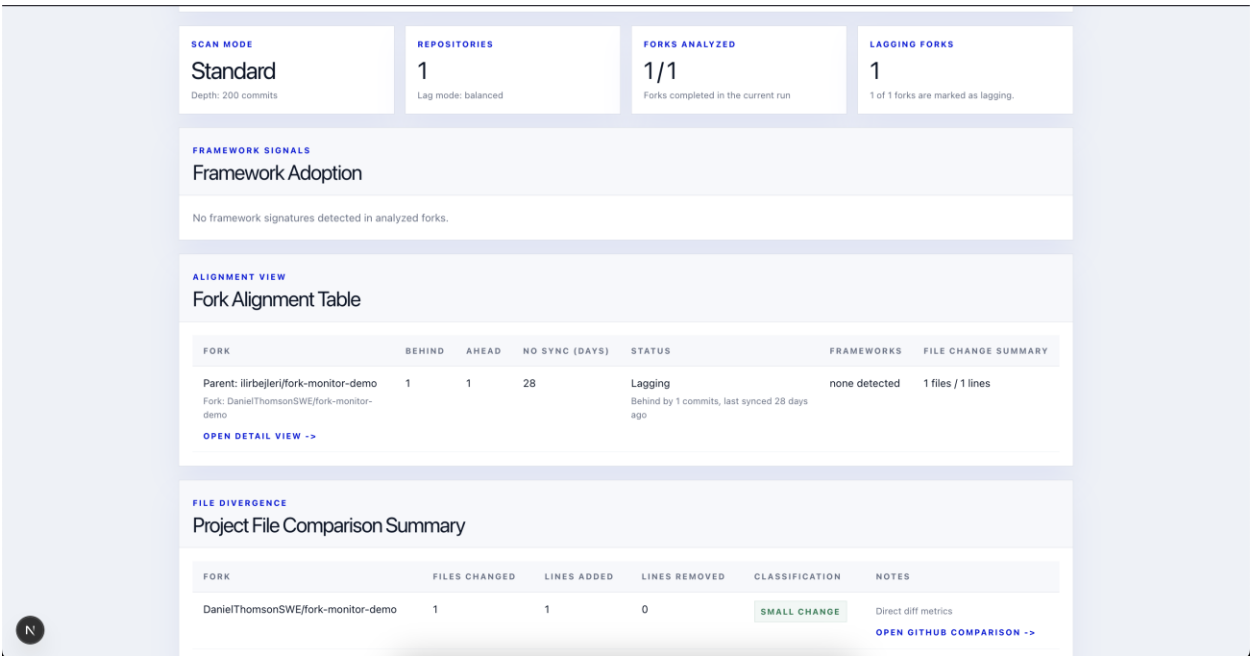


Figure 8: Git Fork Monitor — Scan results with fork alignment status

Figure 8 shows the scan results for four repositories with twelve forks. Each fork row displays the commit distance behind upstream, the number of files changed, the last sync date, detected frameworks, and a color-coded status badge: red for Lagging, yellow for Slightly behind, green for Up-to-date. Clicking a fork row navigates to a detailed comparison page with side-by-side file diffs and commit histories.

Heuristic Analysis

Visibility of System Status: Summary cards at the top provide an at-a-glance overview of the scan. Color-coded status badges make it immediately obvious which forks need attention without reading the numbers.

Match Between System and Real World: The terminology (Behind By, Files Changed, Last Synced) mirrors the language engineers already use when discussing Git repository relationships, reducing the learning curve.

Flexibility and Efficiency of Use: Scan configuration (mode, commit depth, lag thresholds) allows experienced users to tune sensitivity. The Quick/Standard/Deep presets let less experienced users start without configuring anything.

Design Consideration

The visual design of all three tools draws from SLB's corporate brand identity to maintain consistency with the client's existing internal tooling ecosystem. SLB's website and product interfaces use a predominantly light blue and white palette. All tools in this suite mirror this

palette directly, making it feel like a natural extension of SLB's engineering software rather than an external add-on.

Across all three tools, shape and layout conventions are kept consistent. Status indicators always use filled circles (green for healthy/active, red for error/lost, yellow for warning/behind). Data tables use the same column alignment style with muted header rows and alternating row shading for readability. Action buttons (Generate Report, Sign Out, logging toggle) use a high-contrast filled style to distinguish them from navigation elements. Buttons and separators are sharp rectangles which are commonly found on the slb website alongside arrow figures for navigation. These shared conventions reduce the learning curve for engineers who use multiple tools in the suite, even though the tools are deployed independently.

Widget selection prioritizes information density appropriate to each tool's context. The Waveform Monitor uses Chart.js line plots for waveform visualization because signal shape is the primary concern, and tile cards for scalar metrics because engineers need to scan six values at a glance. The PubSub Logger uses a scrolling log table because telemetry data is inherently sequential and timestamped. The Git Fork Monitor uses summary cards for aggregate counts and a sortable data table for fork details because engineers need both a high-level overview and the ability to drill into specific forks.

Appendix

Team Member Contributions

The following summarizes each team member's contributions to the project, including their primary responsibilities, components owned, and contact information.

Iir Bejleri

Contributions: Waveform Analyzer CLI, UI consistency across all tools, dependency reduction for tools on BBB, testing environments using BBB.

Contact: ibejleri3@gatech.edu

Ahmed Alamin

Contributions: Repo Fork Monitor: Includes features such as GitHub Sign In, scanning forked repositories, metrics on fork alignments,

Contact: aalamin3@gatech.edu

Shane Hanley

Contributions: Waveform Analyzer Comparator, Monitoring, Wave Ingestion, Metrics, Visualization, Basic UI, Report Generation, Debugging and Sampling. Optimize project1 pipeline for BeagleBone Black performance, documentation for projects.

Contact: shanley8@gatech.edu

Berk Tunctan

Contributions: XPUB Server implementation, Connecting Project 2 frontend and backend, Stress Testing with nodes

Contact: btunctan3@gatech.edu

Daniel Thomson

Contributions: Design of Project 2 network dashboard design, Testing Project 1 & 2 specifications (RAM & CPU usage) running on BeagleBone Black embedded target device

Contact: dthomson33@gatech.edu